Requirement 5

Design 1

Create class :

a. Interface:
    i. SummonFire
    ii. SummonDirt
b. Concrete Class:
    i. FireState
    ii. IceState
    iii. DirtState
    iv. SplashBehaviour
    v. AttackAction
    vi. FireBlow (weapon)

Pro	**Cons**
Easier to read and understand	Duplication of code inside different states, such as the same method implemented from the DragonState interface duplicated in ice, fire, and dirtState classes.
Each state encapsulates its own logic, which performs its own state logic that follows the single-responsibility principle	Each state defines the magic number that violates the DRY principle and lets future updates be inconsistent

Design 2

Create a new class/interface

a. Interface:
    i. DragonState
    ii. SplashBehaviourInjector

b. Abstract class:

i. StateOfDragon

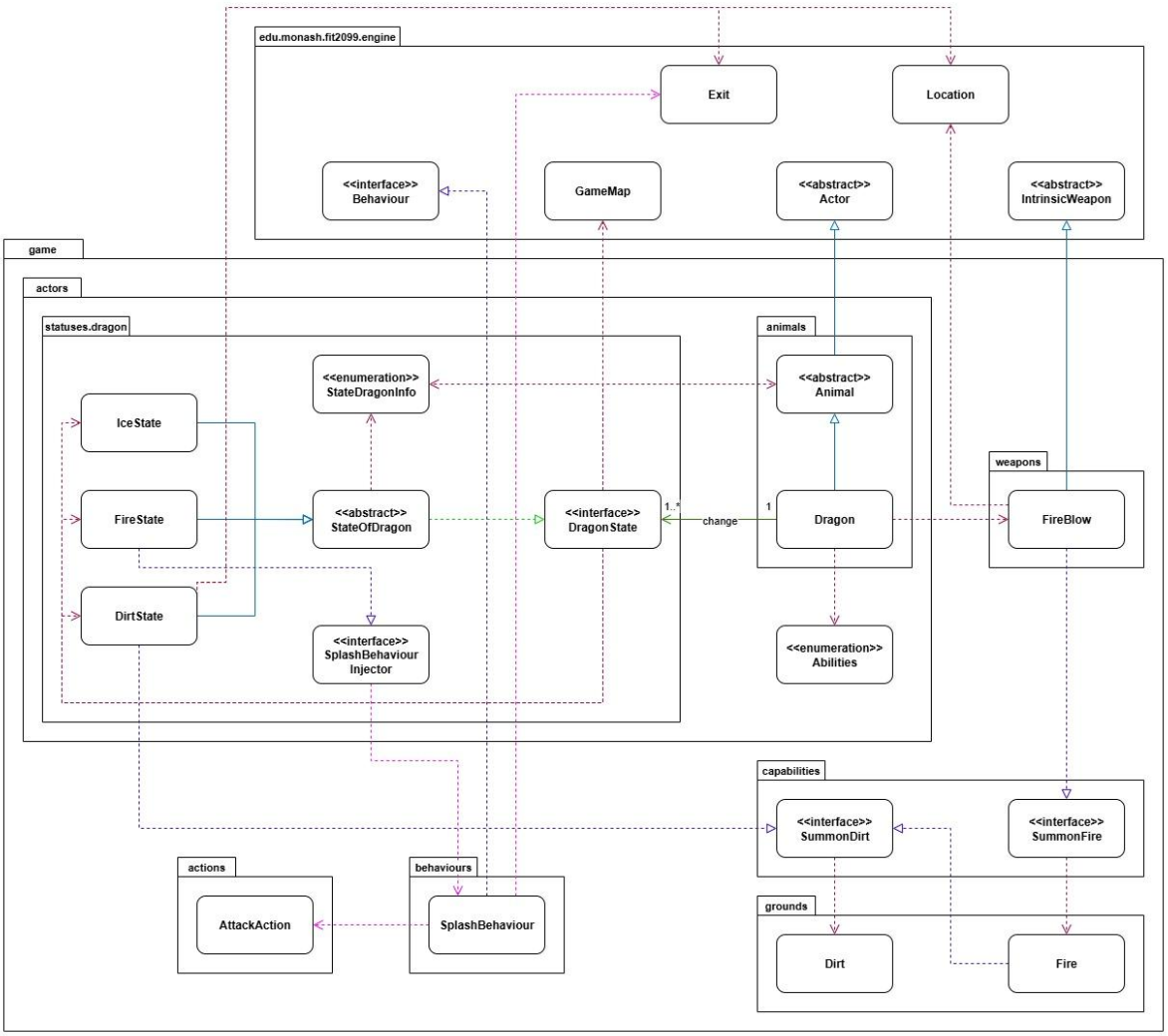
c. Enum class:

i. StateDragonInfo

Pro	**Cons**
Code duplication is reduced because the abstract class handles the shared logic	More complex for beginners
Easier to extend and add a new state (add a Teleport State without modifying the existing code)	
Strong polymorphism, the dragon only deals with the DragonState interface. It doesn't care which state it's in and just calls the methods only.	
Consistent value in an enum that can adjust the duration or probabilities inside the enum class	

Choose:

We suggest design 2 as it was easier to extend, cleaner, and reusable compared to design 1. Regarding the OOP principle, I use an abstract class that ensures both encapsulation and polymorphism are achieved. Each state will share logic, such as duration and specific conditions. Based on the SOLID principle, it makes the code is easier to extend, which follows the Open/Closed principle, and each state follows the single responsibility principle that performs its own state logic. The abstract and enum class follow the DRY principle, which removes the duplicated code compared to the first design.



2. What classes will exist in the extended system
    a. Interface:
        i. DragonState
        ii. SummonFire
        iii. SummonDirt
        iv. SplashBehaviourInjector
    b. Abstract class:
        i. StateOfDragon
    c. Concrete Class:
        i. FireState
        ii. IceState

iii. DirtState

iv. SplashBehaviour

v. AttackAction

vi. FireBlow (weapon)

d. Enum Class:
    i. StateDragonInfo

3. Roles & Responsibilities

    a. Abstract StateOfDragon is an abstract class that implements the DragonState interface that handles the shared logic, such as duration handling, probabilities to the next state. It allows the code to be reusable and consistent across each concrete state class.
    b. DragonState Interface defines all dragon states that enforce the methods for duration, probability, and dragon action.
    c. StateDragonInfo enum stores the probabilities, duration, and detail of each dragon state to centralize all the constants and avoid the magic numbers in the class.
    d. FireState, DirtState, and IceState extend the StateDragon abstract class and implement the specific action and behaviour:
        - FireState has a SplashBehaviour which uses the AttackAction and FireBlow weapon class.
        - IceState can change the surroundings to snow and resistance to warm logic, which doesn't drop the warmth level every turn.
        - DirtState can only change the surroundings to dirt.
    e. The Dragon class extends from the Animal parent class can hold the current state and delegate the behavior to the state object each turn.
    f. FireBlow class enables the AttackAction to use it.
    g. SummonFire interface enables the FireBlow class to handle the burn logic to burn the target location.
    h. SplashBehaviour defines an attack behaviour where the dragon can target a nearby actor and apply damage and fire effect. SplashBehaviourInjector registers this shared behaviour.

4. How these classes relate to and interact with the existing system.

    a. Dragon class extends from Animal and integrates with the game's actor while delegating a state logic to DragonState and ensuring behaviour without modifying the Animal and Actor classes.

b. DragonState is the interface for all dragon states, which ensures polymorphism. Each state defines how the dragon interacts with the GameMap and other actors.
c. StateDragonInfo is the enum class for the StateDragon abstract class to encapsulate the magic number as a constant, such as probabilities to switch state and the duration run for each state
d. StateOfDragon is an abstract class that provides shared behaviour for all states.
e. Each dragon state, such as IceState, FireState, and DirtState class, interacts with the environment by changing the grounds to Snow and Dirt and adjusting the warmth resistance.
f. FireState apply SplashBehaviour through the SplashBehaviourInjector interface.
g. SplashBehaviour extends Behaviour, which allows the dragon to attack the actor using AttackAction by weapons that have a relationship with the Action and Weapon to maintain attack logic.
h. SummonFire enables FireBlow place Fire on a location.
i. SummonDirt enables DirtState to set the grounds as Dirt.
j. Encapsulation and polymorphism occur to ensure the Dragon switch state logic remains isolated from the loop; the actions, actors, and map will interact with Dragon through the Actor interface. These designs follow SOLID and DRY principles by isolating state logic and reducing coupling between the dragon and attack system.

5. How the classes interact to deliver functionality.

a. Dragon interacts with its own DragonState every turns and applies its behaviour to the current state object.
b. DragonState defines required methods for duration and action logic, which ensure each state follows the same structure.
c. StateDragon implements shared logic and each state class extends from this abstract class and overrides specific behaviour.
d. StateDragonInfo enum stores constant values for each DragonState such as duration and probabilities.
e. FireState uses SplashBehaviour to attack the actor via AttackAction enhanced by the FireBlow weapon class.
f. SummonFire applies to the FireBlow to get the burn effects.
g. SplashBehaviourInjector registers the behaviour into states, which ensures the Dependency Inversion Principle, as behaviour can be added without changing the existing class structure.

h. DirtState sets the ground into Dirt, which interacts with SummonDirt.
i. IceState sets the ground into Snow, which interacts with Warmable logic.